# Directed Study Report

## A Study of SparkBWA: Implementation and Experimental Reproduction

Chengkai Yang
Student ID: V01080765

# 1 Introduction

## 1.1 Limitations of Traditional BWA

With the development of next-generation sequencing (NGS) technologies, the scale of DNA sequencing data has increased greatly. Sequence alignment is usually one of the most time-consuming steps in the analysis pipeline, since a large number of short reads need to be mapped to a large reference genome.

Burrows-Wheeler Aligner (BWA) is one of the most widely used tools for sequence alignment. BWA also provides a multi-threaded implementation, which allows users to speed up sequence alignment on personal computers or servers by utilizing multiple CPU cores. However, BWA can only run on a single server and is limited by the CPU and memory resources of that server. When the input sequence files are large, the execution time can be very long, and it may take a significant amount of time to obtain the alignment results.

## 1.2 SparkBWA

To address this limitation, a distributed approach can be used to split the sequence alignment workload across multiple servers and then merge the results at the end. SparkBWA is one implementation based on this idea.

SparkBWA combines the Apache Spark computing framework, the Hadoop YARN resource management mechanism, and the BWA algorithm. It allows BWA to run concurrently on multiple servers and then aggregates the results generated by each node. Spark provides operator-level parallelism and fault tolerance mechanisms, which makes it suitable for this type of data-intensive computation.

In this study, we performed the following work:

1. We upgraded the versions of Spark, BWA, Hadoop, and JDK to recent releases, recompiled the SparkBWA JAR package, and verified that all SparkBWA workflows could run correctly in a local pseudo-Hadoop cluster.

2. We validated SparkBWA on three servers using chr22 and hg38 as reference indices, and resolved issues related to the Hadoop environment, SparkBWA source code, and resource allocation.

3. Using hg38 as the reference genome, we compared the execution speed of SparkBWA with that of BWA.

Experimental results show that under parallel execution on three nodes, SparkBWA achieves a clear performance improvement compared with single-node BWA. Through a series of experiments, this study verifies the feasibility of SparkBWA under the latest versions of BWA and Spark, and provides a analysis of the reasons for performance improvement as well as the limitations of SparkBWA.

# 2 Background

## 2.1 Burrows-Wheeler Aligner (BWA)

Burrows-Wheeler Aligner (BWA) is a widely used tool for short-read alignment. Among its variants, BWA-MEM is currently the most commonly used algorithm, and the experiments in this study are mainly based on BWA-MEM. In this work, we use the lh3-bwa implementation as the baseline for performance comparison.

The BWA-MEM alignment process is mainly CPU-bound. At the same time, the reference index must be loaded into memory during execution. When the index file is large, the entire index needs to be loaded into memory. BWA processes each read in the FASTQ file independently and aligns it using the reference index. Therefore, FASTQ input files are naturally splittable. By specifying multiple threads in `bwa mem`, different portions of the FASTQ file can be processed in parallel on a single machine.

This property of FASTQ files—being splittable and independently alignable against the same index—forms the foundation for running the BWA algorithm in a distributed manner across multiple servers in SparkBWA.

## 2.2 Apache Spark and YARN

Apache Spark is a distributed data processing framework that performs computations mainly in memory. Through the Resilient Distributed Dataset (RDD) abstraction, data is divided into multiple partitions and distributed across cluster nodes, enabling efficient

data-parallel computation. Spark programs can be represented as directed acyclic graphs (DAGs), which makes it easier for users to monitor execution progress.

Spark mainly provides map and reduce style operators, and achieves scalability through data parallelism across multiple nodes. This approach is more scalable than using multi-threading on a single machine. By adding more cluster nodes, parallelism can be increased, avoiding the CPU and memory limitations of single-server execution.

In terms of resource management, SparkBWA does not use Spark Standalone mode. Instead, it runs on YARN, which is responsible for allocating memory and CPU resources. SparkBWA runs executors inside YARN containers and communicates via RPC.

Compared with standalone BWA, SparkBWA also benefits from fault tolerance. If a node runs out of memory or becomes unavailable, tasks can be rescheduled to other nodes, improving the overall stability of the alignment process.

In addition, the original SparkBWA paper points out that reference indices and output files are stored on HDFS. When these files are large, HDFS provides higher I/O throughput than local storage, preventing I/O from becoming a performance bottleneck.

### 2.3 Limitations of Traditional BWA

In summary, SparkBWA addresses the limitations of traditional BWA in two main aspects:

1. **Distributed execution:** Sequence alignment is executed across multiple servers. Even when input data is very large, parallelism can be increased by adding more nodes, overcoming the CPU and memory limits of a single machine.

2. **Resource management and fault tolerance:** With YARN, memory resources can be managed more flexibly, and failures of individual nodes do not cause the entire job to fail.

## 3 SparkBWA Architecture

### 3.1 Overall Workflow

The SparkBWA workflow consists of three main steps: reading and partitioning FASTQ files, invoking BWA to perform alignment, and merging and saving result files. The entire process leverages Spark's distributed computing capabilities while using JNI to call the native BWA algorithm.

First, SparkBWA reads FASTQ files from HDFS and partitions them into multiple partitions distributed across cluster nodes. Since reads are independent, this step naturally enables data-level parallelism. During the map stage, each Spark executor processes

one or more partitions. Each executor caches the FASTQ data locally and invokes the compiled BWA binary via JNI to perform alignment using the local reference index. After alignment, each executor uploads its SAM output file to a temporary directory in HDFS. Finally, SparkBWA merges all SAM files generated by executors into a single SAM file stored in HDFS.

Key design principles, as described in the original paper, include:

1. **No modification of BWA source code:** BWA is treated as a black-box computation module and invoked through JNI, ensuring compatibility with different BWA versions.

2. **JNI-based execution inside YARN containers:** JNI uses off-heap memory, which is still strictly controlled by YARN containers to prevent unexpected memory usage.

### 3.2 Two-Level Parallelism

SparkBWA supports a two-level parallelism model. At the cluster level, Spark map tasks enable parallel execution across nodes by assigning different data partitions to different servers. Within each node, SparkBWA retains BWA's multi-threading capability, allowing multiple BWA threads to run inside each executor and further utilize CPU resources.

However, in practice, we found that two-level parallelism is not always easy to tune. Executor memory configuration and the number of BWA threads must be carefully balanced to achieve optimal performance. In our experiments, improper configurations sometimes caused BWA to fail inside Spark executors. These issues are discussed in later sections.

## 4 System Modernization: SparkBWA on Spark 3.x

### 4.1 Motivation

The original SparkBWA implementation was based on older software versions, including Spark 1.x, Hadoop 2.x, and BWA 0.7.15. Although this environment demonstrated the feasibility of Spark-based sequence alignment at the time, it has become outdated.

In modern production and research environments, Spark 3.x has become the mainstream version, along with newer Hadoop and JDK releases. Therefore, an important goal of this study is to migrate SparkBWA to Spark 3.x and recent Hadoop versions, verify that it can run correctly, and evaluate its performance in a modern distributed environment.

### 4.2 Upgrade Challenges

The main challenges during the upgrade process were related to compatibility, including Spark API changes and JNI native library loading issues. To avoid polluting system environments on servers, Docker was used to build a complete Hadoop and Spark environment locally. Once SparkBWA ran correctly in Docker, the same versions were deployed on physical servers.

A Dockerfile was first used to build the JAR compilation environment, specifying Scala, Spark, and JDK versions. After compilation, it was important to verify that the BWA binary was correctly included in the JAR package.

Next, a full Hadoop and Spark environment was set up on a local laptop using Docker Compose to validate runtime behavior.

After multiple iterations, SparkBWA was successfully upgraded to the following versions:

- Spark: 3.3.2
- BWA: 0.7.19
- JDK: 11
- Hadoop: 3.4.2

SparkBWA was able to run successfully on a single server and produce correct results. Execution logs are available in the GitHub repository.

## 5 Experimental Setup

### 5.1 Cluster Configuration

The distributed experiments were conducted on a small Hadoop cluster consisting of four Linux servers. Each node has 64 GB of memory, with 16 GB allocated to YARN per node. Due to hardware instability, one server (pivla) was sometimes unavailable. Therefore, most experiments were conducted on three servers, providing a total of 48 GB of memory for SparkBWA.

### 5.2 Dataset

We created reference indices based on chr22 and hg38. The FASTQ input files used were `ERR000589_1.filt.fastq` and `ERR000589_2.filt.fastq`.

The hg38 index size is approximately 5.2 GB, and the two FASTQ files are about 1.7 GB in total.

Each Spark executor loads the full reference index into memory when running BWA. Therefore, at least 6 GB of off-heap memory is required for BWA, plus at least 2 GB for the Spark executor itself. As a result, each YARN container requires at least 8 GB of memory. With 16 GB per node, at most two containers can run on each server. Due to hardware limitations, fewer than six containers were used in each experiment.

### 5.3 Baseline

Two baselines were used to evaluate SparkBWA performance:

1. Standalone BWA (version 0.7.19) running on a single machine with 8 threads using the hg38 reference.
2. SparkBWA running on three servers with three containers, also using 8 BWA threads per container.

Execution time was used as the primary performance metric.
A dedicated script was used to submit jobs and measure execution time for both cases.

## 6 Results & Performance Analysis

### 6.1 Overview of Results

After repeated experiments under the same configuration, results show that SparkBWA significantly reduces execution time compared with standalone BWA. On three servers, SparkBWA execution time was approximately one-half to one-third of that of single-node BWA, achieving a performance improvement of about 2 to 3.

Some execution logs are available in the GitHub repository.

### 6.2 Performance Comparison and Analysis

Although three containers were configured, YARN scheduling sometimes placed two containers on the same server, which reduced parallelism. In addition, failure retries were enabled. When the first attempt failed due to memory or other issues, retries increased total execution time. These factors prevented SparkBWA from consistently achieving a full 3 speedup.

Overall, SparkBWA still improved execution speed and enabled more efficient sequence alignment.

### 6.3 Limitations of the Results

It is difficult to precisely quantify performance improvement due to the following reasons:

1. Each container requires approximately 8 GB of memory, limiting the number of containers per node.

2. Optimal configurations depend on index size and dataset size. For smaller indices such as chr22, more aggressive configurations may be possible.

In addition, only execution time was measured. CPU utilization, memory usage, and Spark/YARN overhead were not analyzed in detail.

# 7 Discussion & Insights

## 7.1 Limitations of SparkBWA

Although SparkBWA improves BWA performance, it requires installing Hadoop, YARN, and HDFS, which consume significant system resources. When SparkBWA is not running, these services still occupy memory, reducing available resources for other applications.

## 7.2 Additional Insights

SparkBWA distributes sequence files across nodes, but Hadoop and YARN introduce considerable overhead. An alternative is to use Kubernetes as the resource manager, allowing Spark drivers and executors to run as pods and request resources only when needed.

Similarly, HDFS can be replaced by object storage systems such as S3-compatible storage (e.g., MinIO or cloud object storage services).

# 8 Conclusion & Future Work

In this study, we upgraded SparkBWA to modern software versions and validated its execution using Docker-based environments and a small Hadoop cluster. Results show that SparkBWA can leverage distributed parallelism to improve BWA performance, but configuration tuning is challenging and Hadoop introduces significant overhead.

Future work can focus on improving resource management by replacing YARN with Kubernetes and replacing HDFS with S3-compatible object storage to reduce system overhead.

# 9 Reproducibility

The source code corresponding to this report is publicly available on GitHub:

- GitHub Repository: `https://github.com/chengkaiyang2025/SparkBWA/tree/spark3`

All experiments reported in this study were conducted based on the code and scripts provided in the repository.

### Environment Dependencies

- Operating System: Linux
- Java: JDK 11
- Apache Spark: 3.3.2
- Hadoop / YARN: 3.4.2
- BWA: 0.7.19
- Reference Genome: hg38

## 10 Role of AI in This Study

AI was mainly used for writing Docker-related configuration files and for translating and polishing the English writing of this report.